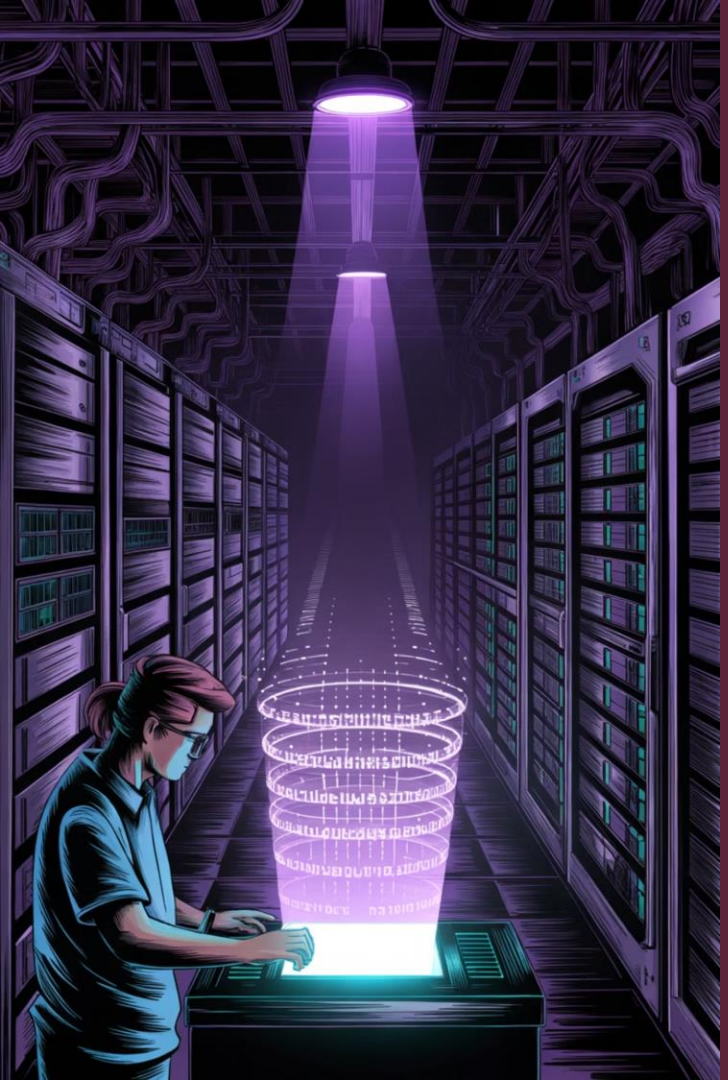


Digital Forensics in the TOR Network

special thanks to *Filippo Goffredo — S322435 ·(former CFCCA student)*

"Tor is a free, open-source anonymity network and software suite. It anonymises TCP-based traffic by routing it through a volunteer-operated overlay network of thousands of relays, applying multiple layers of encryption so that no single node can correlate a user's identity with their network destination. "

TOR — Key Characteristics (1/2)

Low-latency design

Tor is optimised for interactive use (web browsing, SSH), unlike high-latency mixes which sacrifice speed for stronger anonymity.

Onion routing

Traffic is encrypted in multiple layers and routed through exactly three relays: Guard, Middle, and Exit.

Decentralised volunteer network

Over 7,000 relays globally operated by volunteers and organisations, with no central server controlling the routing.

Directory-based consensus

A set of nine hardcoded Directory Authorities (DAs) collectively sign the network consensus - a list of all relays, their capabilities, and flags.

TOR — Key Characteristics (2/2)

Hidden services (onion services)

Tor enables servers to host content reachable only within the Tor network via .onion addresses, providing bidirectional anonymity.

Open-source and audited

The codebase is publicly auditable and has been subject to numerous independent security reviews.

Cross-platform client

Available as the Tor Browser Bundle (Firefox-based) and as a background daemon (tor) for custom configurations.

The Tor Background Daemon

DEFINITION

> *"The tor daemon is a long-running system process —-typically started at boot or on demand — that manages a node's participation in the Tor network. Applications such as the Tor Browser communicate with it via a local SOCKS proxy interface, usually on port 9050."*

What it is

- A core engine that does all the heavy lifting beneath user-facing applications
- Separates the anonymity infrastructure from the applications that use it
- Applications stay Tor-agnostic; all routing logic, cryptography, and circuit management live within the daemon

Why it matters

- A single daemon can serve multiple applications simultaneously
- Applications need no knowledge of the underlying Tor network
- Enables modular architecture: swap or update the daemon independently of the browser

Daemon — Core Roles

Circuit Management

Continuously builds and maintains a pool of pre-constructed circuits. When an application requests a connection, a circuit is ready immediately, reducing perceived latency.

Cryptographic Operations

Handles all onion encryption and decryption: ntor handshakes to establish shared keys with each relay, encrypting outgoing cells in layers, and decrypting incoming layers. The application has no awareness of this.

Directory Client

Fetches and caches the network consensus from Directory Authorities and mirror relays. Maintains an up-to-date map of relays, their bandwidth, flags (Guard, Exit, HSDir, etc.), and public keys.

Guard Node Persistence

Maintains and persists the user's selected guard nodes across sessions (stored in the state file), ensuring continuity of the guard selection strategy even after restarts.

Daemon — Interfaces & Advanced Features

SOCKS Proxy Interface

Exposes a SOCKS4/SOCKS5 interface on 127.0.0.1:9050. Any application configured to use it routes its TCP connections through Tor automatically, without needing to know anything about the underlying network.

Control Port

Exposes a control interface on port 9051 by default. Allows other processes to send commands and receive events — e.g. requesting a new circuit, querying relay statistics, or monitoring connection state. Tor Browser uses this for the "New Identity" button.

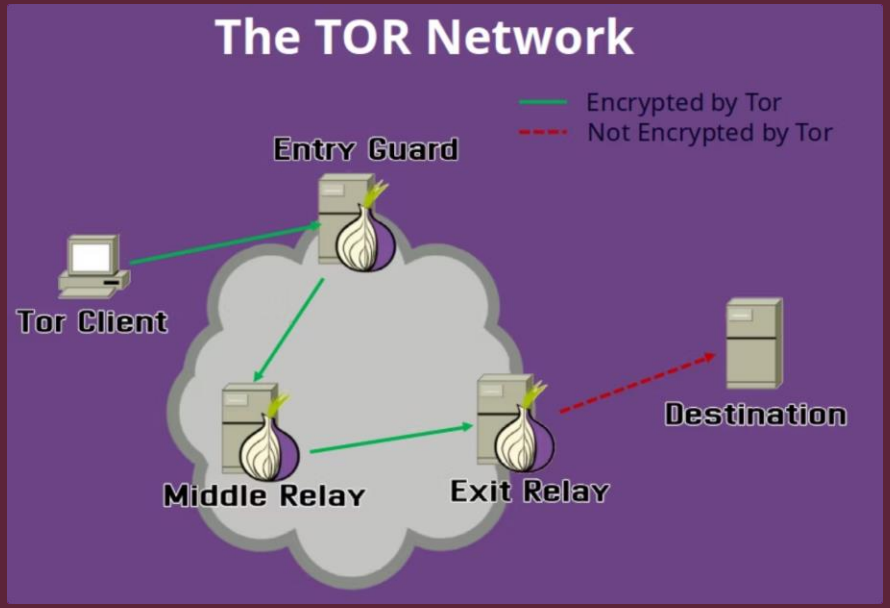
Relay Operation

When configured as a relay or exit node, the daemon accepts inbound circuits from other Tor clients, forwards cells, and manages bandwidth accounting and rate limiting.

Bridge & Pluggable Transport Coordination

In censored environments, manages connections to bridge relays and coordinates with pluggable transport processes (obfs4proxy, meek) that disguise Tor traffic as ordinary HTTPS or other protocols.

The TOR Network



How Traffic Flows

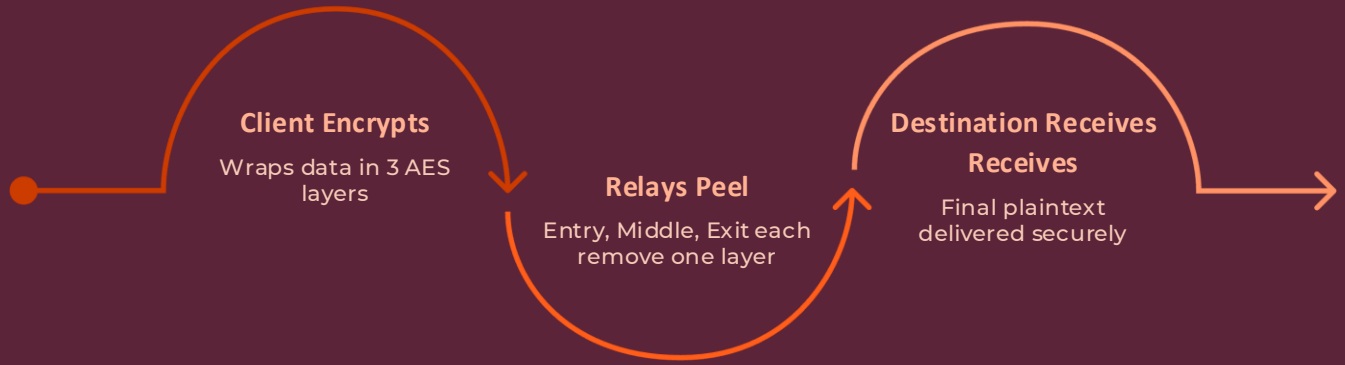
Traffic is routed from the **Tor Client** through three sequential relays — **Entry Guard**, **Middle Relay**, and **Exit Relay** — before reaching the destination.

> Traffic inside the Tor network is **encrypted**. Only the segment from Exit Relay to Destination is **not** encrypted by Tor.

No single relay knows both the origin and destination of the data, ensuring structural anonymity.

The TOR Network — Layered Encryption

Data is wrapped in **multiple layers of encryption bef**ore transmission. Each relay decrypts exactly one layer, like peeling an onion, so no relay ever has full visibility of the communication path.



Each encryption layer corresponds to a distinct relay key. The client negotiates separate symmetric keys with each relay via ECDH, ensuring cryptographic isolation between hops.

Cryptographic Workflow in TOR

When a Tor client (Alice) wishes to connect to a server, it constructs a three-hop circuit through a precise cryptographic process:

01

Guard Node Selection

The client selects a long-term guard node, establishing a TLS connection. A Tor handshake (ntor or TAP) is performed, yielding a shared symmetric key, `K1`, for secure communication.

02

Middle Node Extension

The client tunnels a `CREATE` cell through the guard node to a middle relay. This step establishes a second symmetric key, `K2`. The guard node can only identify the client and the middle relay.

03

Exit Node Extension

A third `CREATE` cell is tunneled through the guard and middle relays to an exit node, establishing the final symmetric key, `K3`. At this stage, the exit relay only knows the middle relay and the intended destination server.

04

Circuit Establishment

The complete three-hop circuit is now formed: Alice → Guard (`K1`) → Middle (`K2`) → Exit (`K3`) → Destination. Each hop uses a unique symmetric key for end-to-end encryption and anonymity.

Guard Node

The Entry Point

The **guard node** is the **first relay** in a user's Tor circuit. It is the **only relay** that knows the user's real IP address — but it has no knowledge of the final destination.

This deliberate separation ensures that downstream relays cannot correlate a user's identity with their browsing activity.

Correlation Attack Mitigation

Limits adversarial entry points, reducing traffic correlation risk.

Stability by Design

Guard nodes are assigned for weeks/months, providing reliable, consistent circuits and mitigating long-term traffic analysis

Mitiganting long-term traffic analysis

Every time a circuit is built, the entry relay sees the user's real IP address. If Tor picked a random entry relay for every circuit, an adversary operating or monitoring many relays would get repeated chances to be the entry point. Over time, the probability that the adversary at least once lands as the guard approaches 1.

> This is the intersection attack: even if the adversary can't win on any individual circuit, they win cumulatively by accumulating observations across many circuits.

Without Persistent Guards

- Each new circuit is an independent trial
- After n circuits, the probability of the adversary never being the entry node is $(1 - p)^n$, which decays exponentially toward zero
- A patient adversary just waits

With Persistent Guards

- The adversary's chance of controlling the guard is fixed at the moment of selection — roughly proportional to the guard's share of guard-flagged bandwidth
- That probability doesn't grow over time, no matter how many circuits are built
- If the guard is honest, the user is protected permanently for that guard's tenure

Guard Nodes — How Assignment Works in Practice

1	Guards are chosen from relays assigned the Guard flag in the consensus, requiring sustained uptime (at least 8 days), sufficient bandwidth, and a stable history. This prevents adversaries from spinning up short-lived relays to catch users during guard rotation.
2	The client selects a small set (currently 1 primary guard in modern Tor, previously 3). The smaller the set, the fewer relays ever learn the user's IP, shrinking the exposure surface.
3	A guard is kept for roughly 3 months (or until it goes offline or loses its Guard flag). During this window, the real IP is exposed only to that one relay - not to the broader relay population.
4	The daemon records selected guards in its state file on disk. This survives restarts, reboots, and browser updates, ensuring continuity. A fresh restart doesn't accidentally trigger a new guard selection.
5	A guard is retired only if it becomes unreachable for a sustained period, loses the Guard flag, or its tenure expires. If the primary guard goes temporarily offline, Tor uses a secondary guard rather than immediately selecting a new one, preventing churn-based attacks.

Guard Nodes — Limitations

Adversarial Guard

If the assigned guard is adversarial, the user has the same exposure as without guards — the guard knows the IP and the circuit's first hop. The scheme reduces the probability of this happening; it doesn't eliminate it.

Traffic Correlation

A traffic correlation adversary who can observe both the guard's traffic and the exit node's traffic can still deanonymise the user, regardless of how long the guard has been kept.

Network-Level Adversaries Adversaries

Guard persistence helps against relay-level adversaries. It does not help against network-level adversaries who observe links rather than controlling nodes.

> *"Persistent guards convert an adversary's attack from 'keep trying until you get lucky' into 'you either got lucky at the start or you didn't' "*

TOR Cryptographic Algorithms

RSA

Key exchange and authentication between the client and entry node during circuit setup.

AES

Symmetric encryption of data in transit. Each relay peels one AES layer as traffic passes through.

ECDH

Efficient key exchange between relays. Particularly suited to resource-constrained environments.

SHA-256

Data integrity checking and hashing throughout the protocol.

PKI

Verifies the authenticity of relays, preventing malicious nodes from infiltrating the network.

TOR Cryptographic Primitives

Curve25519 Diffie-Hellman

Used in the ntor handshake for forward-secure key exchange, resistant to passive decryption even if long-term keys are later compromised.

AES-128-CTR

Fast, stream-cipher mode encryption for cell payloads, efficient and parallelisable.

Ed25519 Signatures

Used for relay identity and consensus signing, providing strong authentication with small key sizes.

SHA-3 / BLAKE2b

Used in v3 onion service key derivation, replacing the deprecated SHA-1 used in older protocols.

Each circuit uses ephemeral session keys derived via the ntor handshake. Compromise of a relay's long-term identity key does not expose past session traffic — a critical property for protecting historical communications.

TOR — Cell Structure and Encryption

Tor traffic is 512-byte fixed-size cells (or variable-length cells for v3 onion services).

fixed sizing to prevent traffic analysis based on packet length, which could otherwise reveal information about the communication.

Layer		key used	visibility
Exit Node	(outermost)	K3	Plaintext of the destination TCP stream (after decryption)
Middle Node		K2	Encrypted blob — cannot read content or destination
Guard Node		K1	Encrypted blob — cannot read content or destination

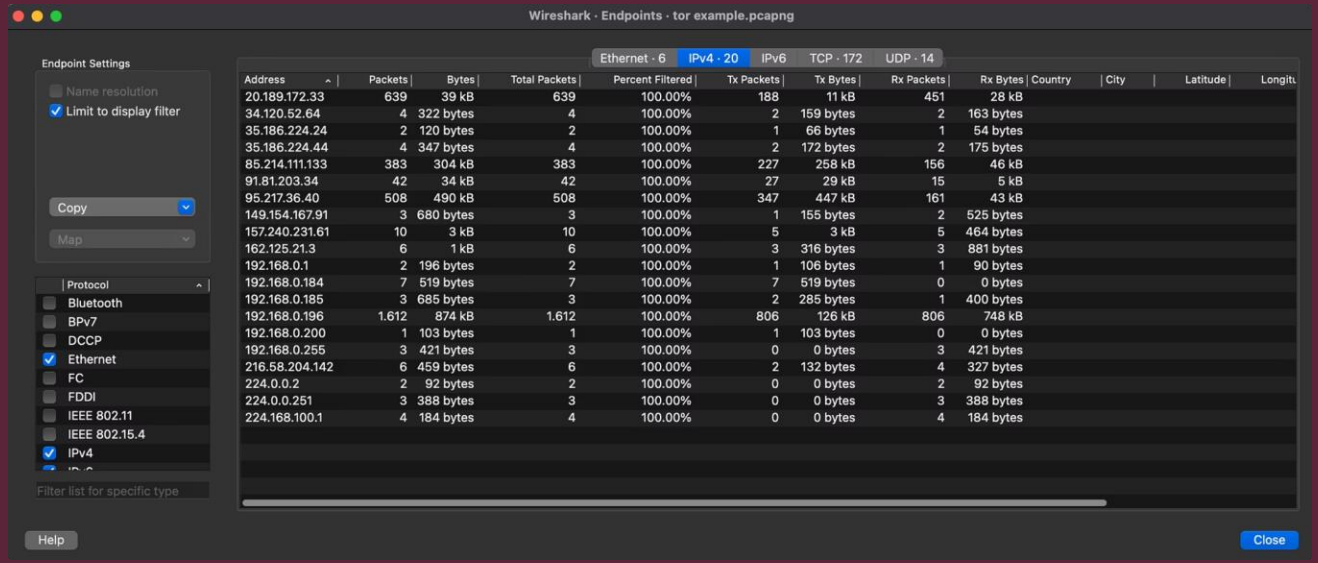
Encryption is applied in reverse order at the client, (`K3` encrypting the innermost layer, `K1` the outermost).

Each relay then strips its respective layer using AES-128 in Counter Mode (CTR).

`Digest` field in the cell header allows each relay to verify data integrity without fully decrypting the remaining layers.

TOR Demo - Capturing Traffic

A forensic analyst captures live network traffic using Wireshark. The resulting packet list contains a mix of IP addresses from various sources. At first glance, no relay labels are visible — the raw data does not explicitly flag TOR-associated IPs.



To identify TOR connections, analysts must cross-reference captured IPs against an authoritative **TOR Node List**.

TOR Demo — Identifying Relays

85.195.253.142	lodrich	9001	-	FGHRSDV	3,430,823	Tor 0.4.8.12	max(at)freifunk-dresden(dot)de [tor-relay.co]
85.195.253.142	lodrich2	9005	-	FGHRSDV	3,434,622	Tor 0.4.8.10	max(at)freifunk-dresden(dot)de [tor-relay.co]
85.197.44.109	Unnamed	587	-	FRDV	65,185	Tor 0.4.8.12	mika@baldpapa.de
85.2.82.41	hawk	9001	-	FRSDV	261,067	Tor 0.4.8.12	hawk(at)swiss-enigma(dot)ch
85.201.79.190	JJRelay	443	-	FHRSDV	389,113	Tor 0.4.8.10	alienate.too@gmail.com
85.203.39.232	Havssnigel	39541	-	RDV	34	Tor 0.4.8.12	email:m736368726569ner[]i2pmail.org proof:uri-rsa abuse:m736368726569ner[]i2pmail.org pgp:C8E426879D40DC7792C5E504C6E610C626929
85.204.116.100	LaryMi	23007	-	FRV	1,350,145	Tor 0.4.8.12	LaryMi87@gmail.com
85.208.144.164	porte	443	-	FGHRSDV	4,334,861	Tor 0.4.8.12	tor(at)alo(dot)is [tor-relay.co]
85.208.69.66	swordsman	9001	-	FGRSDV	176,414	Tor 0.4.8.12	swordsman @ satoshi
85.209.46.236	guidemenow	9001	-	FHRSDV	584,062	Tor 0.4.8.10	
85.209.49.222	rubefeli	443	-	FGHRSDV	2,120,494	Tor 0.4.8.12	A8F9764C rubefeli <"relay nickname" AT pm.me>
85.209.51.37	GermanCraft6	443	-	FGHRSDV	4,261,149	Tor 0.4.8.12	ContactInfo email:knight AT germancraft dot net url:germancraft.net proof:dns-rsa abuse:knight AT
85.214.111.133	SupportSeaWatch	443	-	FGHRSDV	11,959,555	Tor 0.4.8.12	0xa1<at:>mailbox<dot:>org
85.214.149.151	alittlehelper	9091	-	FGHRSDV	5,454,616	Tor 0.4.8.12	tortech123456 <tortech123456 at protonmail com>
85.214.200.184	kol3v14	443	-	FGHRSDV	11,599,556	Tor 0.4.8.10	kol3v14@protonmail.com
85.214.202.158	Schwabentor	9001	-	FGHRSDV	12,214,900	Tor 0.4.8.12	
85.214.58.236	highwaytohoell	9001	-	FRSDV	11,660,535	Tor 0.4.8.12	Check http://highwaytohoell.de
85.215.114.1	Hodor	443	-	FGHRSDV	907,564	Tor 0.4.8.12	torB22196@gmail.com
85.215.129.4	LeClassiqueRelay	42069	-	FHRSDV	451,403	Tor 0.4.8.12	luis.carilla1@gmail.com
85.215.136.64	0xdeadbeef	9001	-	FGRSDV	3	Tor 0.4.8.12	Hurdy-Gurdy <admin AT my-mail dot rocks>
85.215.138.207	ketatrade4391	443	-	FGHRSDV	2,138,294	Tor 0.4.8.12	darmstadt@fridaysforfuture.de
85.215.145.68	Hattorren	9001	-	FGHRSDV	4,615,490	Tor 0.4.8.12	<olafuringvason@posteo.is>
85.215.153.202	magic848relay	9001	-	FRDV	7,303,060	Tor 0.4.8.9	tor-operator@mufff.in
85.215.158.17	MRWHITE	443	-	RDV	68,767	Tor 0.4.8.12	nope@nopey.comd
85.215.160.111	supercomputer	44441	-	FGRSDV	1,743,355	Tor 0.4.8.12	Mafra01(AT)Safe-mail(DOT)net
85.215.160.128	eclipse19	443	-	FGHRSDV	2,164,026	Tor 0.4.8.12	freiraum_TK (AT) protonmail (DOT) ch
85.215.181.146	PommespanzerDE3	443	-	FGHRSDV	936,052	Tor 0.4.8.12	relay // at // pommespanzer // dot // cc
85.215.181.200	cylonhelps1	9000	-	FGHRSDV	7,632,276	Tor 0.4.8.10	ja@cylon.ovh

TOR Node List Cross-Reference

Forensic analysts consult a comprehensive **TOR Node List** —a regularly updated registry of known relay and exit node IP addresses. By automating a lookup against this list, any matching address in the capture is flagged as a TOR relay.

In this demo, IP address **85.214.111.133** was identified as a TOR relay by cross-referencing the captured flow against the node list.

This process is easily automatable via scripting, making large-scale forensic analysis highly practical.

TOR Demo - Bridge Evasion

Bypassing Known Node Detection

The Tor Browser can be manually configured with a bridge in its settings, routing traffic through unlisted entry points that do not appear in public TOR Node Lists.

This significantly complicates forensic detection, as bridge IPs are not publicly documented and must be discovered through other investigative techniques.

⚠ Bridge configurations represent a substantial challenge for analysts relying solely on node list cross-referencing.

Tor's threat: the 14 Eyes Alliance

The Five Eyes alliance — United States, United Kingdom, Canada, Australia, and New Zealand — has expanded into the broader Fourteen Eyes, adding nine more countries including Germany, France, Sweden, and the Netherlands.

This coalition enables cross-border intelligence sharing, substantially narrowing the anonymity guarantees of TOR for users within member jurisdictions.

concrete instantiation of the "global passive adversary"

⊗ More than **60% of all TOR nodes** fall under the jurisdiction of 14 Eyes member countries.

14 Eyes — Implications for TOR TOR Users

Law enforcement agencies do not need to control every relay in a user's circuit. By observing traffic patterns and behavioral anomalies across nodes within their jurisdiction, member states can collectively infer suspicious activity — even without decrypting individual packets.

Pattern Analysis

Statistical correlation of timing and volume across multiple jurisdictions.

Node Jurisdiction

Relay operators in member states are subject to legal compulsion to cooperate.

Cross-Border Intelligence

Shared data enables multi-hop traffic correlation without single-nation dominance.

How the Guard Node Attack Works

Sends undetectable relay drop cells to mimic legitimate traffic. Uses timing analysis to identify relays carrying the highest traffic load.

Traffic is spread evenly across circuits via a round-robin method. Prolonged attacks (24+ hours) expose statistical anomalies, identifying the Hidden Service Guard node with high confidence.

Challenges: false positives, complex traffic patterns, network fluctuations. Success factors: sustained attack duration, effective statistical analysis, and adequate resource availability.

Compromises Tor's anonymity by exposing the HS Guard node. Used in real-world law enforcement operations (likely including the BoysTown takedown)

Source: "Dropping on the Edge: Flexibility and Traffic Confirmation in Onion Routing Protocols"— Rochet & Pereira

BoysTown Overview

A landmark real-world case illustrating the forensic application of guard node analysis against a criminal darknet platform.

BoysTown was a darknet child exploitation forum with over 400,000 registered users, operating exclusive Ricochet chatrooms for member communication. German authorities — the BKA and the Public Prosecutor General's Office in Frankfurt — successfully dismantled the platform.

Investigators likely began by analyzing chatroom traffic and then performed a guard discovery attack.

BoysTown administrators had not deployed Vanguards - the Tor add-on protecting against such attacks pinning also other middle relays as the initial guard node.

operational security failures, especially in communication, remain the most reliable path to criminal identification.

further readings: https://www.heise.de/en/news/Boystown-investigations-Catching-criminals-on-the-darknet-with-a-stopwatch-9904534.html